SERVA

MVP Development Roadmap

From Prototype to Working Product

A complete build guide for a founder-led development approach

April 2026 — Confidential

1. Honest Assessment: Building It Yourself

You said you want to build this yourself, and you have a $75K–$200K budget. That combination is actually powerful — but let's be direct about what "build it myself" realistically looks like for a platform this complex.

What Serva's MVP Actually Requires

Serva is not a simple app. It's a two-sided marketplace with real-time scheduling, payment processing, custom form building, search and discovery, and separate interfaces for consumers, businesses, and platform admins. That's three distinct applications sharing a backend. Here's what's involved:

Component	Complexity	Estimated Effort
Consumer mobile app (iOS + Android)	High	400–600 hours
Business dashboard (web)	High	300–500 hours
Admin panel (web)	Medium	150–250 hours
Backend API + database	High	400–600 hours
Payment system (Stripe Connect)	High	150–250 hours
Real-time booking engine	Very High	200–300 hours
Search & discovery	Medium	100–150 hours
Notifications (push, email, SMS)	Medium	80–120 hours
Custom intake form builder	Medium–High	120–200 hours
DevOps, testing, deployment	Medium	150–250 hours
TOTAL		2,050–3,220 hours

At a full-time pace (40 hrs/week), that's 12–18 months for a single experienced developer. For someone learning while building, it could be 18–30 months.

The Recommended Approach: Founder + Targeted Help

The smartest play with your budget isn't "build every line yourself" or "outsource everything." It's a hybrid:

- **You own the product and core logic:** You learn the tech stack, build key features, make architecture decisions, and understand every part of the system. This makes you a technical founder, which is invaluable for fundraising and hiring.

- **You hire specialists for the hard parts:** Payment integration, real-time systems, DevOps, and mobile deployment have deep gotchas that take years to learn. Hiring an experienced freelancer for 2–4 weeks of targeted work on these saves you months of pain.
- **You use AI heavily:** Claude, Cursor, GitHub Copilot — modern AI coding tools can 3–5x your output on boilerplate, components, database queries, and API endpoints. Use them aggressively.

Budget allocation: ~$30K–$50K on freelance specialists and infrastructure. ~$10K–$15K on tools, services, and subscriptions. The remaining budget is your runway (living expenses while you build).

2. MVP Tech Stack

The technical architecture document we built earlier is the long-term vision. For the MVP, we simplify dramatically. The goal is speed-to-market with a foundation that scales.

Recommended Stack

Layer	Technology	Why for MVP
Mobile App	React Native + Expo	One codebase for iOS & Android. Expo handles builds, updates, and deployment. You skip native dev entirely.
Web Apps (Biz + Admin)	Next.js + TypeScript	Full-stack React framework. Server-side rendering for SEO on business profiles. API routes eliminate a separate backend for simple endpoints.
Backend API	Next.js API Routes + tRPC	Start with Next.js API routes. No separate server to deploy and manage. tRPC gives type-safe API calls between frontend and backend.
Database	Supabase (PostgreSQL)	Hosted Postgres with built-in auth, real-time subscriptions, file storage, and row-level security. Free tier is generous. Eliminates 80% of backend setup.
Auth	Supabase Auth	Email/password, Google, Apple sign-in out of the box. No custom auth code needed.
Payments	Stripe Connect	Industry standard for marketplace payments. Handles split payments, payouts, compliance. Non-negotiable.
Search	Supabase Full-Text + PostGIS	PostgreSQL has excellent full-text search and geo queries built in. Skip Elasticsearch until you need it.
File Storage	Supabase Storage	Business photos, intake form uploads. S3-compatible, built into your database platform.
Email	Resend	Modern email API. Beautiful transactional emails. Simple pricing. React Email for templates.
SMS	Twilio	Appointment reminders and verification codes. Industry standard, pay-per-message.
Push Notifications	Expo Notifications	Built into Expo. Free. Works on both iOS and Android.
Hosting	Vercel	Deploy Next.js with zero config. Auto-scaling, global CDN, preview deployments. Free tier covers MVP.
Monitoring	Sentry	Error tracking and performance monitoring.

		Free tier is generous. Catch bugs before users report them.

Why this stack? It maximizes what you get for free. Supabase replaces a custom backend, custom auth, custom file storage, and real-time infrastructure. Vercel eliminates DevOps. Expo eliminates native mobile complexity. You focus on product logic, not infrastructure.

Monthly Cost Estimate (MVP)

Service	Tier	Monthly Cost
Supabase (database, auth, storage, real-time)	Pro	$25
Vercel (hosting, CDN, serverless)	Pro	$20
Stripe (payment processing)	Pay-as-you-go	2.9% + 30¢/txn
Resend (transactional email)	Free → Pro	$0–$20
Twilio (SMS reminders)	Pay-as-you-go	~$0.0079/SMS
Expo (mobile builds)	Free → Production	$0–$99
Sentry (error tracking)	Developer	$0–$26
Apple Developer Account	Annual	$8.25/mo ($99/yr)
Google Play Developer	One-time	$2.08/mo ($25 once)
Domain + DNS (serva.com or similar)	Annual	$1–$15/mo
TOTAL (pre-revenue)		~$80–$215/mo

3. Your Learning Path

If you're not already a developer, here's the most efficient learning path to get you building Serva. This is not a "learn to code from scratch" curriculum — it's a targeted, project-driven path focused specifically on the skills Serva requires.

Phase 0: Foundations (Weeks 1–4)

You need to be comfortable with JavaScript/TypeScript, React, and basic database concepts before touching any of the Serva-specific stack.

1. JavaScript Fundamentals — Complete a focused JS course (freeCodeCamp, The Odin Project, or Scrimba). Focus on: variables, functions, arrays, objects, promises/async-await, and ES6+ syntax. Skip the DOM manipulation — React handles that.
2. TypeScript Basics — TypeScript adds type safety to JavaScript. Take the official TypeScript tutorial (typescriptlang.org). You don't need to master generics and advanced types. Focus on: type annotations, interfaces, and basic type inference.
3. React Fundamentals — Build 2–3 small projects using React. The official React tutorial (react.dev) is excellent. Focus on: components, props, state (useState), effects (useEffect), and forms.
4. SQL Basics — Take SQLBolt (sqlbolt.com) or the Supabase SQL tutorial. You need: SELECT, INSERT, UPDATE, DELETE, JOINs, and basic WHERE clauses.

Phase 1: The Serva Stack (Weeks 5–8)

1. Next.js — Follow the official Next.js tutorial (nextjs.org/learn). Build a small project with routing, server components, and API routes. This is the framework everything runs on.
2. Supabase — Follow the Supabase quickstart for Next.js. Set up a project, create tables, write queries, set up auth. Build a simple CRUD app (a to-do list or contact manager).
3. React Native + Expo — Follow the Expo tutorial (docs.expo.dev). Build a small mobile app. If you know React, you're 80% of the way there — the concepts are nearly identical.
4. Stripe — Follow the Stripe Connect quickstart for platforms/marketplaces. Understand PaymentIntents, connected accounts, and the onboarding flow.

Phase 2: Build While Learning (Weeks 9+)

At this point, you start building Serva. You'll learn the remaining skills (real-time subscriptions, complex queries, form builders, etc.) by building the actual features. Every feature you build for Serva teaches you the skill you need.

Time investment: If you're starting from zero coding experience, expect 8–12 weeks of focused learning (20–30 hrs/week) before you're productive on the real build. If you have some coding background, you can compress this to 4–6 weeks.

4. Phased Build Plan

This is the exact order you should build Serva's MVP. Each phase produces something testable and builds on the last. Do not skip ahead.

Phase 1: Foundation & Data Layer (Weeks 1–3)

Set up the project, database, and core data models. Nothing visible to users yet — this is the skeleton.

- **Set up repos:** Create a monorepo with three apps: /apps/web (Next.js for business dashboard + admin), /apps/mobile (Expo for consumer app), /packages/shared (shared types, utilities, validation schemas).
- **Database schema:** Create all tables in Supabase: users, businesses, services, staff, bookings, availability_rules, reviews, form_templates, form_submissions, categories. Set up row-level security policies.
- **Authentication:** Implement Supabase Auth with email/password, Google, and Apple sign-in. Create user roles (consumer, provider, admin). Build login/signup screens for both web and mobile.
- **Seed data:** Write a seed script that creates 20–50 fake businesses across multiple categories with services, pricing, and availability. This gives you real data to build against.

Milestone: You can sign up as a consumer or business owner, and you have a populated database to query.

Phase 2: Business Dashboard — Provider Experience (Weeks 4–7)

Build the business-facing web app first. Why? Because you need businesses on the platform before consumers. And building the provider tools first means the data exists for the consumer app to consume.

- **Business profile creation:** Onboarding flow where a provider enters business name, category, address, description, photos, and operating hours. Display a live preview of how their profile will look.
- **Service menu builder:** CRUD interface for adding services with: name, description, price, price type (fixed/starting at/hourly), duration, buffer time, deposit requirement.
- **Calendar & availability:** Weekly recurring availability rules (e.g., Monday–Friday 9am–5pm). Date-specific overrides (closed on holidays, extended hours). Staff assignment if multiple staff members.

- **Intake form builder:** Drag-and-drop form builder where providers create custom intake forms. Field types: text, textarea, dropdown, date picker, checkbox, file upload. Preview mode to test the form.
- **Booking management:** View incoming bookings, confirm/decline, see client intake submissions, and mark as completed or no-show.

Milestone: A real business owner can create a complete profile, list services with pricing, set their availability, build intake forms, and manage bookings.

Phase 3: Consumer Mobile App — Core Booking Flow (Weeks 8–12)

Now build the consumer experience — the flow shown in your interactive prototype.

- **Home screen:** Category grid, search bar, featured/nearby businesses. Location permission request for geo-based results.
- **Search & discovery:** Full-text search across business names, service names, and categories. Filter by category, distance, rating, and price range. Results sorted by relevance/distance.
- **Business profile:** Full profile view matching the prototype: hero image, business info, rating, reviews, tabbed interface (Services, Reviews, Photos, Info).
- **Service selection:** Tap a service to see details and begin booking.
- **Calendar booking:** Date picker showing available dates. Time slot grid pulled from the provider's real-time availability. Slot locking (when a user starts checkout, hold the slot for 10 minutes to prevent double-booking).
- **Intake form:** Render the provider's custom intake form. Pre-fill known fields (name, email, phone) from the user's profile. Validate required fields before proceeding.
- **Payment:** Stripe payment sheet integration. Save card for future bookings. Show order summary with service fee. Process payment.
- **Confirmation:** Confirmation screen with booking details. Push notification scheduled for 24-hour and 1-hour reminders. Add-to-calendar deep link.

Milestone: A real consumer can search for a business, view the profile, select a service, pick a time, fill out the intake form, pay, and receive a confirmation. The core product loop is complete.

Phase 4: Reviews, Notifications & Polish (Weeks 13–15)

- **Reviews:** After a completed booking, prompt the consumer to leave a star rating and text review. Display reviews on business profiles. Allow business owners to respond.

- **Notifications:** Push notifications for booking confirmations, reminders (24h and 1h), cancellations, and new reviews. Email confirmations and receipts via Resend. SMS reminders via Twilio (optional but high-impact for reducing no-shows).
- **Booking management (consumer):** View upcoming and past bookings. Cancel or reschedule. View receipts and payment history.
- **Cancellation policies:** Configurable by business: free cancellation window, late cancel fee, no-show charge. Automatic refund processing through Stripe.

Milestone: The consumer experience is complete and polished. Users get reminders, can manage bookings, and the review loop creates trust.

Phase 5: Admin Panel & Launch Prep (Weeks 16–18)

- **Admin dashboard:** Platform analytics (total bookings, revenue, active users, new signups). Business approval/moderation queue. User management. Review moderation (flag/remove inappropriate content).
- **Business verification:** Manual or automated verification flow for new business signups. Display “Verified” badge on approved profiles.
- **Content policies:** Terms of service, privacy policy, cancellation policy templates. Implement report/flag functionality.
- **Performance & testing:** Load testing, security audit (especially around payments), accessibility review, app store screenshot preparation.
- **App store submission:** Submit to Apple App Store and Google Play Store. Build the public-facing website (serva.com) with landing page, business signup, and app download links.

Milestone: The platform is ready for real users. Admin tools let you manage the marketplace. Apps are live in both stores.

5. Where to Hire Specialists

These are the specific areas where hiring an experienced freelancer for 2–4 weeks will save you months:

Area	What They Do	Est. Cost	Where to Find
Stripe Connect integration	Set up marketplace payments, onboarding, payouts, refunds, webhooks	$3K–$8K	Toptal, Upwork (Stripe-certified developers)
Real-time booking engine	Slot locking, conflict resolution, concurrent booking handling	$3K–$6K	Toptal, Arc.dev (look for marketplace experience)
DevOps / deployment	CI/CD pipeline, staging environment, monitoring setup, security hardening	$2K–$5K	Toptal, Infrastructure Club
UI/UX design refinement	Polish the prototype into production designs, design system, icon set	$3K–$8K	Dribbble, Toptal, ADPList
App Store submission	Handle the review process, metadata, screenshots, compliance	$1K–$3K	Upwork, Fiverr Pro
Security audit	Penetration testing, auth review, payment security verification	$3K–$8K	Synack, HackerOne, Cobalt

Total estimated specialist spend: $15K–$38K. This is the highest-leverage use of your budget — each of these specialists does in 2 weeks what would take you 2–3 months to figure out alone.

6. Complete Timeline

Period	Phase	Key Deliverables	Milestone
Weeks 1–8	Learn the stack	Complete foundations, Next.js, Supabase, Expo, Stripe tutorials	You can build a full-stack app with auth and payments
Weeks 9–11	Phase 1: Foundation	Monorepo, database schema, auth, seed data	Users can sign up; database is populated
Weeks 12–15	Phase 2: Biz Dashboard	Provider profiles, service menus, calendar, intake forms, booking mgmt	A business can fully set up on Serva
Weeks 16–20	Phase 3: Consumer App	Home, search, profiles, calendar, intake, payment, confirmation	Core booking loop works end-to-end
Weeks 21–23	Phase 4: Polish	Reviews, notifications, booking management, cancellations	Consumer experience is complete
Weeks 24–26	Phase 5: Admin + Launch	Admin panel, verification, testing, app store submission	Platform is live in app stores
Weeks 27–30	Beta launch	Invite 50–100 local businesses, 500+ consumers for beta testing	Real users, real feedback, real bookings

Total timeline: approximately 7–8 months from starting to learn, to having a live beta with real users. If you already have coding experience, subtract 4–8 weeks from the learning phase.

7. Budget Breakdown

Item	Cost Range	Notes
Freelance specialists (Stripe, real-time, DevOps, design, security)	$15K–$38K	Highest-leverage spend
UI/UX designer (production design system)	$3K–$8K	Polish the prototype for production
Legal (ToS, privacy policy, business entity)	$3K–$7K	Get this right from day one
Infrastructure & services (7 months)	$600–$1,500	Supabase, Vercel, Stripe, etc.
Apple + Google developer accounts	$124	One-time costs
Domain name (serva.com or similar)	$500–$5,000	Depends on domain availability
Development tools & subscriptions	$500–$1,500	Cursor, Claude Pro, Figma, etc.
Beta launch marketing (local)	$2K–$5K	Social ads, local outreach, flyers
Contingency (15%)	$4K–$10K	Unexpected costs and scope changes
TOTAL HARD COSTS	$29K–$77K	

The remainder of your $75K–$200K budget is your personal runway. If you're building full-time for 7–8 months, you need to cover living expenses during that period. The lower your hard costs, the longer your personal runway lasts.

8. Essential Tools & Resources

Development Tools

- **Code Editor:** Cursor (AI-powered VS Code fork) — $20/month. The AI autocomplete and inline chat will dramatically accelerate your coding. This is the single most valuable tool for a solo developer.
- **Claude Pro:** $20/month. Use Claude for architecture decisions, debugging, code reviews, writing complex queries, and generating boilerplate components.
- **GitHub:** Free. Version control, code hosting, and GitHub Actions for CI/CD.
- **Figma:** Free tier works fine. For reviewing and exporting the production designs your UI/UX freelancer creates.

Learning Resources

- **React:** react.dev (official docs — free, excellent)
- **Next.js:** nextjs.org/learn (official interactive tutorial — free)
- **Supabase:** supabase.com/docs + their YouTube channel (free)
- **React Native:** reactnative.dev + docs.expo.dev (free)
- **Stripe Connect:** stripe.com/docs/connect (free, with sandbox for testing)
- **TypeScript:** typescriptlang.org/docs + Matt Pocock's free tutorials
- **SQL:** sqlbolt.com (free, interactive)

Key Libraries You'll Use

- **Tailwind CSS:** Utility-first CSS framework. Style your app by adding classes directly to elements. Fastest way to build polished UIs.
- **shadcn/ui:** Beautiful, accessible React components built on Tailwind. Buttons, forms, modals, tables — all pre-built.
- **React Hook Form + Zod:** Form handling and validation. Critical for the intake form builder and all form-heavy interfaces.
- **TanStack Query:** Data fetching and caching. Makes your app feel instant by managing server state intelligently.
- **date-fns:** Date manipulation library. Essential for calendar logic, time slot calculations, and scheduling.
- **Zustand:** Lightweight state management. For global app state like the current user, cart, and booking flow.

9. MVP Success Criteria

Your MVP is successful when you can demonstrate the following:

Technical Milestones

- **End-to-end booking:** A consumer can search, find a business, select a service, pick a time, fill out an intake form, pay, and receive confirmation — all without leaving the app.
- **Real payments:** Stripe processes a real transaction, splits the platform fee, and deposits funds to the business owner's bank account.
- **Real-time accuracy:** Two consumers cannot book the same time slot. Availability updates instantly when a booking is made or cancelled.
- **Both app stores:** The consumer app is live and installable from both the Apple App Store and Google Play Store.
- **Business autonomy:** A business owner can sign up, build their complete profile, list services, set availability, create intake forms, and manage bookings without any help from you.

Business Milestones (Beta Launch)

- **50–100 businesses:** Active provider profiles across at least 8–10 different service categories in your launch city.
- **500+ consumers:** Active users who have completed at least one booking.
- **Repeat bookings:** At least 20% of users make a second booking within 30 days.
- **Revenue:** Real transaction revenue flowing through the platform, even if small. This proves the model works.
- **Feedback loop:** Active feedback channels with both providers and consumers. A clear list of what to build next based on real user needs.

This MVP — with real users, real payments, and real data — is what makes the $3.5M seed round in your pitch deck credible. Investors fund traction, not ideas.

10. Your Next Steps

Here's exactly what to do this week to start building Serva:

1. Set up your development environment: Install Node.js, VS Code (or Cursor), Git, and the Expo CLI. Create accounts on GitHub, Supabase, Vercel, and Stripe.
2. Secure the brand: Register the domain name (check serva.com, getserva.com, joinserva.com, servaapp.com). Set up social media handles on Instagram, Twitter/X, LinkedIn, and TikTok.
3. Start learning: Begin Phase 0 of the learning path. JavaScript fundamentals first. Set a goal of 2–3 hours per day, 5 days per week.
4. Form the business entity: Register an LLC or C-Corp (C-Corp if you plan to raise venture capital). Talk to a startup attorney about equity structure.
5. Bookmark this roadmap: This document is your compass. Revisit it at the start of each phase to stay on track. Adjust timelines as you learn your actual velocity.

Go build it, Jed.

The prototype exists. The plan exists. Now make it real.